

1 Objective

The intent of this handout is to give a overview of modern transformer architectures. Tokenization is omitted here. The core algorithms, i.e. BPE, haven't changed much.

2 Architecture, Then vs. Now

In contrast to the original transformer, which was an encoder-decoder model for machine translation, the predominant architecture now is decoder-only: a sequence to sequence transformation from token IDs to probability distributions at each position. In these decoder-only, auto-regressive models:

1. Token IDs are passed through a linear layer to get token embeddings $[..., seq, d_m]$
2. Token embeddings are passed through transformer blocks $[..., seq, d_m]$
3. Transformed embeddings are passed through another linear layer, to project the embedding at each position to vocabulary size $[..., seq, d_{vocab}]$
4. The softmax is applied to go from logits to output probabilities. The value at the i 'th index on a single embedding is the probability of i 'th token appearing next in sequence.

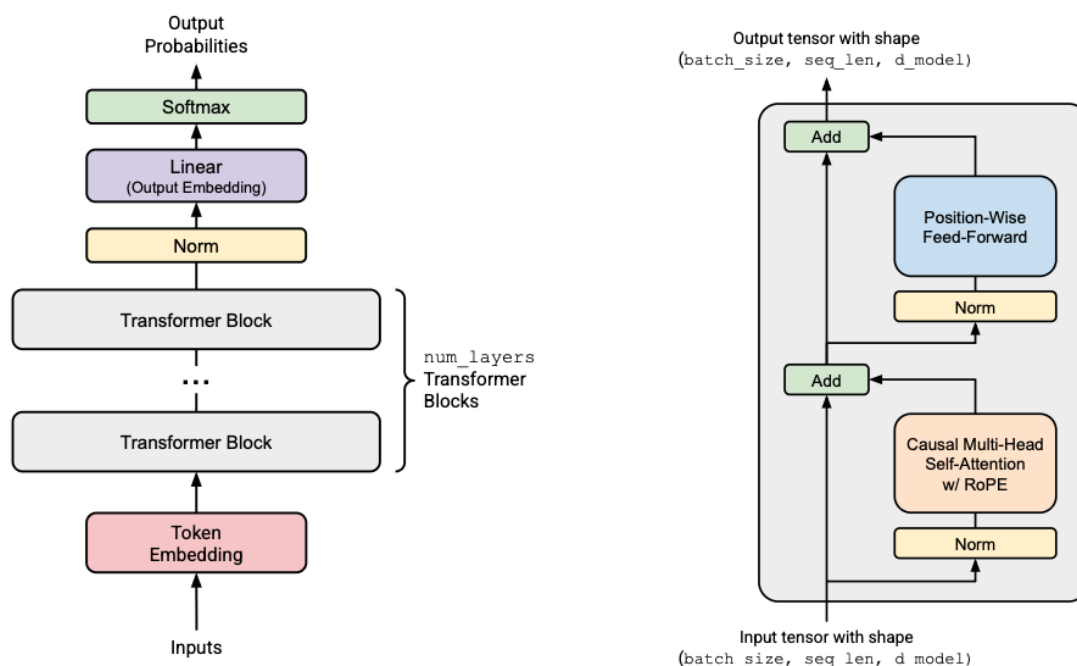


Figure 1: A picture of a modern decoder-only, pre-norm transformer

In a single transformer block, we have the following components (explained more later):

1. **Norm block**: scales each input embedding, for training stability.
2. **MHSA+RoPE**: obtain a contextual embedding by attending to embeddings before.
3. **FFN**: A linear transformation and nonlinear activation to add non-linearity.
4. **Residual Streams**: Note that the input is always added to the output

In recent years, we can see convergence towards the pre-norm architecture shown in Fig. 1, RMSNorm, and SwiGLU activation. For example, the Llama, Qwen, and DeepSeek model families share the same architectural components. Following *CS336: Language Modeling from Scratch*¹, I implemented a 22M transformer² with this architecture and **probed the impact of these components on model training and validation losses**.

Model	Year	Norm	Parallel	Pre-norm	Pos. Emb.	Act.
Original transformer	2017	LN	Serial			ReLU
GPT	2018	LN	Serial		Abs	ReLU
GPT3 (175B)	2020	LN	Serial	✓		Sinc
GPT-J	2021	LN	Parallel	✓	RoPE	GELU
GPT-NeoX	2022	LN	Parallel	✓	RoPE	GELU
PaLM (540B)	2022	RMS	Parallel	✓	RoPE	SwiGLU
Mistral (7B)	2023	RMS	Serial	✓	RoPE	SwiGLU
LLAMA (65B)	2023	RMS	Serial	✓	RoPE	SwiGLU
LLAMA2 (70B)	2023	RMS	Serial	✓	RoPE	SwiGLU
Qwen (14B)	2024	RMS	Serial	✓	RoPE	SwiGLU
DeepSeek (67B)	2024	RMS	Serial	✓	RoPE	SwiGLU
Yi (34B)	2024	RMS	Serial	✓	RoPE	SwiGLU

Table 1: Evolution of transformer architectures. LN for LayerNorm, RMS for RMSNorm.

Below are the hyperparameters of the 22M model. d_{ff} refers to the feed-forward dimension and is typically 8/3 of the d_{model} . Beta1 and Beta2 values are standard for AdamW.

- `tokenizer_vocab_size`: 10000 (trained on TinyStories)
- `context_length`: 256
- `d_model`: 512
- `d_ff`: 1344
- `rope_theta`: 10000
- `num_layers`: 4
- `num_heads`: 16
- `batch_size`: 64
- `beta1`: 0.9
- `beta2`: 0.999
- `eps`: 10^{-8} (in denominators for stability)
- `weight_decay`: 0.01
- `total_tokens_processed`: 327,680,000 (TinyStories)

¹<https://stanford-cs336.github.io/spring2025/>

²<https://github.com/ml-ryanlee/llm-hwk1-basics>

3 Norms – for training stability

As seen in Fig. 2, a normalization layer is absolutely necessary for numerical stability, which is probably why it appears everywhere in the modern transformer. I implemented RMSNorm:

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad \text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$$

In the original transformer architecture, a LayerNorm was used, which applies a standard normalization that we'd think of: normalizing each embedding (normalizing over d_{model}) to have a mean at 0 and variance equal to 1. We can view RMSNorm as a simpler version of layer norm, where we skip the mean centering and approximately normalize variance. Practitioners have found RMSNorm to be just as effective as LayerNorm and faster too.

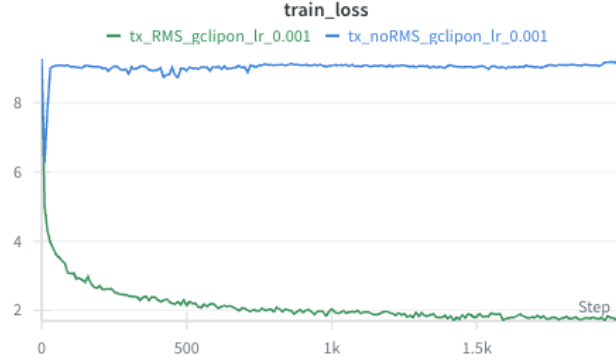


Figure 2: Without RMSNorm, the model cannot effectively train

4 Causal Multi-head Self Attention & Positional Embeddings

The most expensive operation in a modern transformer is the causal multi-head self attention block. To break down what is happening, let's first consider **regular causal self-attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \quad \text{where } Q = W_Q x, K = W_K x, V = W_V x.$$

The mask $M \in \mathbb{R}^{n \times m}$ ensures each position can only attend to itself and previous positions:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases} \quad \text{e.g.} \quad M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}.$$

At each position, we get a contextual embedding by taking a weighted sum of embedding values, where the weight is determined by *similarity* between the current position embedding and previous (loosely defined by Q-K product). Note that in numerator of softmax, we have e^z , thus adding -inf means zeroing out the weight for a future token embedding pair.

To improve the expressivity of self-attention, we extend regular causal attention by slicing our Q,K,V vectors into n_{heads} segments, then attention is applied independently for each segment. To recombine the segment products, we take a *linear combination* of the heads by applying a learned output matrix W_O . **Multi-head self attention (MHSA)** is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)$$

$$\text{where } \text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{with } Q_i, K_i, V_i \text{ being slice number } i \in \{1, \dots, h\}$$

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x)$$

Attention, although appealing for its parallelization (every token attends to every other previous token and itself, in parallel), has a weakness. The attention product, a weighted sum, has not been obtained by taking into account any information about the relative position of the token in the sequence to the tokens it attended to.

The modern solution that is most widely adapted is **Rotary Position Embeddings (RoPE)**: we deterministically rotate the Q and K vectors of a given token embedding based on the position of the embedding in the sequence. What does this do? Imagine your input sequence consisted of a single token, repeated N times. Without RoPE, the Q vector and K vector would be exactly the same, and thus the attention weight calculated between every one of the tokens would also be the same. With RoPE, tokens farther away will be less similar than tokens closer together, as they are more rotated away.

$$R_k^i = \begin{bmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{bmatrix} \quad R^i = \begin{bmatrix} R_1^i & 0 & \dots & 0 \\ 0 & R_2^i & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & R_{d/2}^i \end{bmatrix} \quad \theta_{i,k} = \frac{i}{\Theta^{(2k-1)/d}}$$

The key property of RoPE is that the attention score depends on relative position:

$$Q_i^T K_j = (R^i q)^T (R^j k) = q^T (R^i)^T R^j k = q^T R^{j-i} k$$

where the identity $(R^i)^T R^j = R^{j-i}$ is applied. We rotate our key vector by $(j-i)/\Theta$.

We can observe in Fig. 3 that RoPE allows for faster convergence and lower losses.

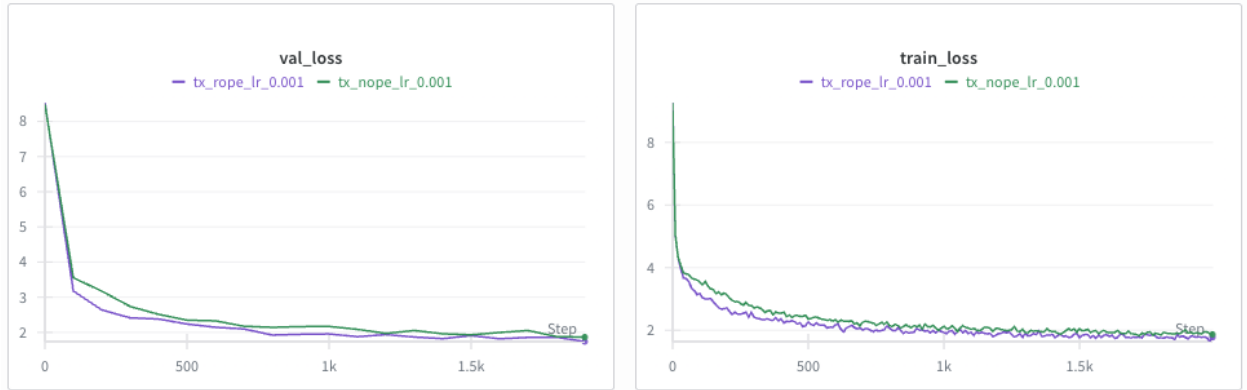


Figure 3: RoPE shows a slight performance improvement vs. its ablation NoPE

5 FFN

After multi-head self-attention, we apply a feed-forward network, which consists of a linear transformation (matrix multiply) and a non-linear activation. While attention allows tokens to communicate and aggregate information, the FFN provides additional expressiveness by including non-linear transformations that help the model learn complex patterns. Historically, this has been added to early transformers used the ReLU activation for the feed-forward activation: now it's **Swish-Gated Linear Unit (SwiGLU)**. Admittedly, I don't have a strong intuition as to why this is an improvement over other activations. However, we can note that:

- the SiLU/Swish activation is differentiable everywhere, vs. ReLU with a hard cut-off.
- A SwiGLU FFN has “gating”, i.e. the nonlinear activation is scaled, which could help with vanishing gradients.

Putting the SiLU/Swish and GLU together, we get the SwiGLU FFN:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1x) \otimes W_3x), \quad \text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

where $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and canonically, $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$.

How important or impactful is that gating component $\text{SiLU}(W_1x) \otimes W_3x$?

We can see in Fig. 4 that there is an improvement from using the gated unit. SwiGLU is used in modern large language models including LLaMA, PaLM, and others, having largely replaced ReLU and GELU activations in state-of-the-art architectures.

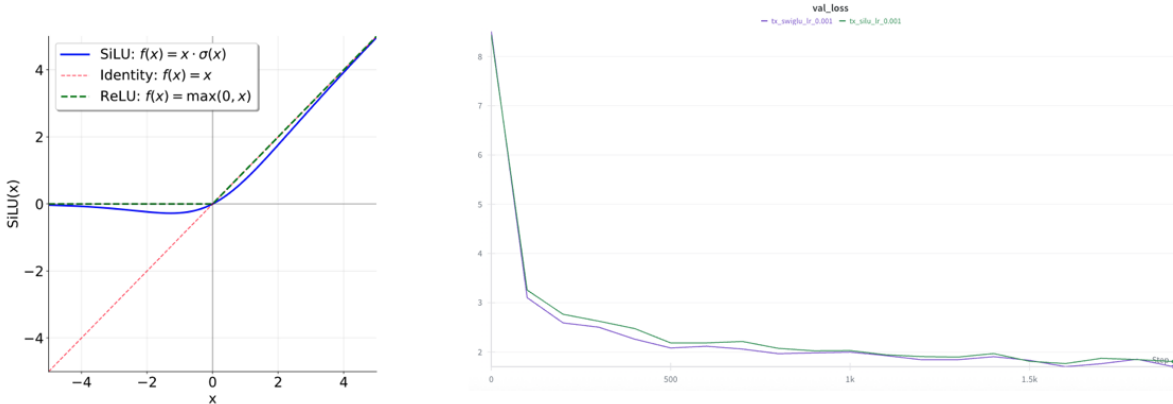


Figure 4: The SiLU activation function. SwiGLU with gating performs better than w/o.

Generation from the Mini Transformer (Just for Fun)

Input: Once upon a time

Generated:, there was a little bird named Tim. Tim loved to soar high in the sky. One day, Tim saw a little bird on the ground. The bird was sad and needed help to fly. Tim wanted to help the bird. He looked around and finally saw the bird's nest.

6 Residual Stream

How important is it to separate the norm from the residual? As it turns out, it makes quite a difference. Researchers have found that **separating norms from the residual stream** improves performance, but why?

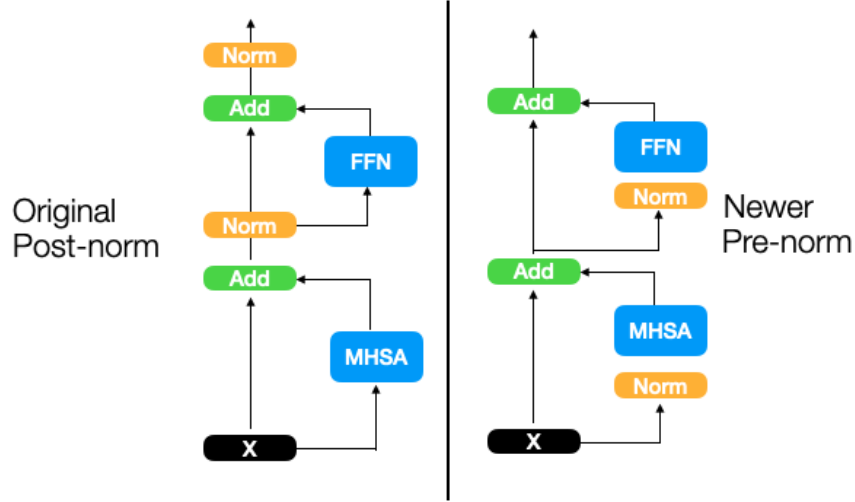


Figure 5: Post-norm in early architectures vs. pre-norm in modern architectures.

Why Pre-Norm Works Better: The key difference lies in gradient flow. For post-norm:

$$y = \text{Norm}(x + F(x)) \implies \frac{\partial y}{\partial x} = \frac{\partial \text{Norm}}{\partial (x + F(x))} \cdot \left(I + \frac{\partial F}{\partial x} \right)$$

Gradients must pass through the normalization Jacobian, which can scale them unpredictably. For pre-norm:

$$y = x + F(\text{Norm}(x)) \implies \frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial \text{Norm}(x)} \cdot \frac{\partial \text{Norm}}{\partial x}$$

The identity term I provides a direct gradient path that bypasses normalization entirely, preventing vanishing or exploding gradients even in very deep networks. As shown in Figure 6, pre-norm architecture substantially outperforms the post-norm architecture.

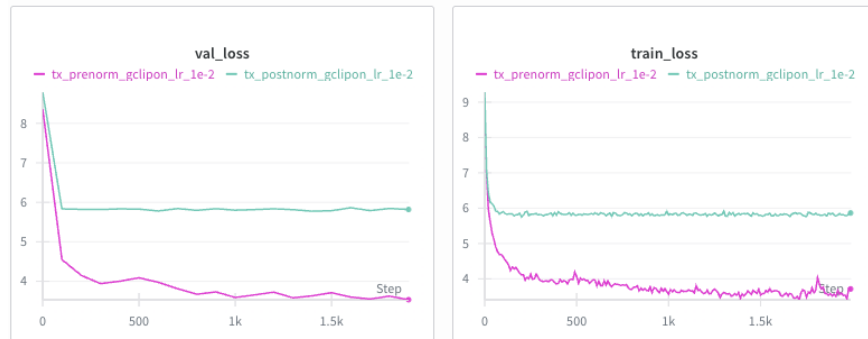


Figure 6: Pre-norm architecture outperforms original post-norm architecture.